

Complete Technical Requirements Document (TRD): Zero-Cost Ephemeral Chat

This TRD outlines the technical implementation, constraints, and architecture for building and running an ephemeral, non-persistent, 2-user real-time chat application entirely on free-tier infrastructure.

System Architecture & Free-Tier Blueprint

- **Frontend Client:** React Native via Expo (free local testing via Expo Go or web build via Vercel).
- **Backend Application Server:** Node.js + Express + Socket.io deployed on **Render Free Web Service** (750 free instance hours/month).
- **Identity Database: Neon Serverless PostgreSQL** (0.5 GiB free storage, scales to zero when idle).
- **Real-Time Memory Routing:** Process RAM (Map<username, socketId>) for single-instance, zero-cost message passing; no external Redis instance required.
- **Persistent Chat Storage: None.** Database contains zero message tables, zero chat logs, and zero chat metadata.

Data Schema (PostgreSQL)

Run the following SQL migration on Neon to initialize the credentials table:

```
CREATE TABLE IF NOT EXISTS accounts (  
  id SERIAL PRIMARY KEY,  
  username VARCHAR(32) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE INDEX IF NOT EXISTS idx_accounts_username ON  
accounts(username);
```

- No foreign keys, no message schemas, and no message archive tables exist anywhere in the database.

Authentication & Token Flow

- **Registration:**
 - Accepts username (alphanumeric, 3–32 chars) and plain-text password (minimum 8 chars).
 - Hashes password using bcryptjs with 10 salt rounds.
 - Writes record to accounts.
- **Authentication:**
 - Validates password against password_hash.
 - Generates a signed JSON Web Token (HS256, 7-day expiry) containing payload {

id, username }.

- **Socket Handshake:**

- Client sends JWT in the auth payload during the WebSocket handshake (socket = io(SERVER_URL, { auth: { token } })).
- Backend verifies token via jwt.verify. Connections without valid tokens are terminated immediately.

WebSocket Event Protocol & Lifecycle

Connection Mapping

- The server maintains an internal in-memory map: const activeUsers = new Map(); mapping username -> socket.id.
- On connection: activeUsers.set(socket.user.username, socket.id).
- On disconnect: activeUsers.delete(socket.user.username).

Event Contracts

Event Name	Direction	Payload Structure	Behavior
direct_message	Client \rightharpoonarrow Server	{ to: string, text: string }	Server checks activeUsers.get(to). If online, routes packet. If offline, drops packet immediately.
message_received	Server \rightharpoonarrow Client	{ from: string, text: string, timestamp: number }	Client receives message and pushes to local React state array in RAM.
user_status	Client \rightharpoonarrow Server	{ targetUsername: string }	Checks if activeUsers.has(targetUsername) and emits boolean status back.

Ephemeral Data Retention Rules

- **Server Memory:** Messages exist solely as transient Node.js buffers in memory during transport. No message logging or caching is enabled.
- **Database Layer:** Zero write operations triggered on message events.
- **Client Layer:**
 - Message history is kept exclusively in component state (useState([])).
 - Closing the browser tab, killing the app process, or refreshing the view wipes all prior conversations permanently.

Free-Tier Implementation Constraints & Mitigations

- **Render Spin-Down:** Free Render services spin down after 15 minutes of inactivity. Initial startup can take 30–50 seconds.
 - *Mitigation:* Use a free cron service (e.g., Cron-Job.org) to ping GET /health every 10 minutes, or accept the cold start.
- **Single Instance Limit:** In-memory socket routing requires all users to be on the same Node.js instance. Since the app runs on a single free Render container, in-memory

routing functions without needing a Redis cluster.

- **Neon Auto-Suspend:** Neon suspends the database compute when inactive. The initial login after dormancy will incur a ~1-second latency while the compute wakes up.

Security Specifications

- **Transport Encryption:** Enforced HTTPS and WSS via Render's automatic TLS management.
- **Rate Limiting:** In-memory rate limiting applied to /register and /login endpoints using express-rate-limit (max 10 requests per 15 minutes per IP) to prevent credential brute-forcing.
- **Input Sanitization:** Usernames restricted to regex `^[a-zA-Z0-9_]{3,32}$`. Messages restricted to max 2,000 characters.

Do you want the step-by-step terminal commands and code files to deploy this to Render and Neon?